APACHE WEB SERVER

Task 1: Install Apache and PHP

Step 1: Install the required packages

Open your Linux terminal and switch to root:

su -

Install Apache and PHP:

dnf install httpd php -y

Start and enable Apache:

systemctl enable --now httpd

Check the service:

systemctl status httpd

Look for active (running).

Step 2: Create the PHP webpage

Create the file:

vi /var/www/html/index.php

Press i and enter this code:

```
<!DOCTYPE html>
<html>
<head>
   <title>My Foodie Blog</title>
</head>
<body>
   <h1>Welcome to My Foodie Blog</h1>
</body>
</html>
```

Save the file by pressing Esc, typing :wq, and pressing Enter.

Step 3: Set the correct SELinux context

On RHEL, use:

restorecon -v /var/www/html/index.php

Step 4: Test PHP

Run:

php -v

Then test Apache locally:

curl http://localhost/index.php

Open your browser and visit:

http://SERVER_IP/index.php

Replace SERVER_IP with your Linux server's IP address. You can find it with:

ip addr

You should see Welcome to My Foodie Blog in the browser. If you see raw PHP source instead, stop and tell me; PHP is not being processed correctly.

Task 2: Configure a custom 404 error page

Step 1: Create the error page

Run:

vi /var/www/html/404.html

Enter:

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Not Found</title>
</head>
<body>
  <h1>Oops! This page is as empty as your cart.</h1>
  <p>Looks like this page went out for food!</p>
  <a href="/">Return to Foodie Blog</a>
</body>
</html>
```

Save using Esc, :wq, and Enter.

Step 2: Configure Apache

Open the Apache configuration:

vi /etc/httpd/conf/httpd.conf

Go to the end of the file and add:

ErrorDocument 404 /404.html

Save and validate the configuration:

httpd -t

Expected output:

Syntax OK

Restart Apache:

systemctl restart httpd

Step 3: Test the custom page

Run:

curl -i http://localhost/this-page-does-not-exist

You should get an HTTP 404 Not Found response containing:

Oops! This page is as empty as your cart.

You can also open this URL in your browser:

http://SERVER_IP/this-page-does-not-exist

Task 3: Set up an FTP server and upload an image

We'll use vsftpd for this lab.

Step 1: Install and start FTP

dnf install vsftpd ftp -y

systemctl enable --now vsftpd

Verify:

systemctl status vsftpd

Look for active (running).

Step 2: Create an FTP user

useradd -m ftpblog

passwd ftpblog

Set a password when prompted. You'll use this username and password in your FTP client.

Step 3: Configure vsftpd

Back up the original configuration:

cp /etc/vsftpd/vsftpd.conf /etc/vsftpd/vsftpd.conf.bak

Edit it:

vi /etc/vsftpd/vsftpd.conf

Ensure these settings are present and not commented out:

anonymous_enable=NO

local_enable=YES

write_enable=YES

local_umask=022

For a simple lab, configure the FTP user's home directory as the web root and restrict the user to that directory:

usermod -d /var/www/html ftpblog

Set access permissions without changing ownership of the entire web root:

setfacl -m u:ftpblog:rwx /var/www/html

For this simple local lab, add the following settings to vsftpd.conf:

chroot_local_user=YES

allow_writeable_chroot=YES

Security note: Plain FTP transmits credentials without encryption. Use this only in an isolated lab network; for real systems, prefer SFTP or explicitly configured FTPS. Do not expose this FTP service to the public internet.

Restart the service:

systemctl restart vsftpd

If SELinux prevents FTP writes, check the audit logs rather than disabling SELinux.

Step 4: Allow FTP through the firewall for the lab

If you need to connect from another machine, allow the FTP control service:

firewall-cmd --permanent --add-service=ftp

firewall-cmd --reload

Standard FTP also needs a configured passive-port range for reliable data transfers through a firewall. If your FTP client cannot list directories or transfer files, we can configure passive mode next.

Step 5: Upload an image using an FTP client

Use an FTP client such as FileZilla on your client machine.

Connect using:

- Host: Your server IP address
- Protocol: FTP
- Username: ftpblog
- Password: The password you set
- Port: 21

Upload a sample image, such as food.jpg, to the FTP user's home directory, which is /var/www/html.

Verify on the Linux server:

ls -l /var/www/html/food.jpg

Set the normal web-content SELinux context:

restorecon -v /var/www/html/food.jpg

Then open:

http://SERVER_IP/food.jpg

If the image displays, the FTP upload and web-server access are working.

Task 4: Run a Node.js playlist service on port 3000

We'll create a simple API that returns mock music playlist data in JSON format.

Step 1: Install Node.js and npm

Run:

dnf install nodejs npm -y

Verify installation:

node --version

npm --version

If the packages aren't available, check that your RHEL repositories are enabled.

Step 2: Create the Node.js application

Create a directory:

mkdir -p /opt/now-playing

cd /opt/now-playing

Create the application file:

vi /opt/now-playing/app.js

Paste this code:

```
const http = require('http');

const playlist = {
   app: "Now Playing",
   playlist: [
      {
         title: "Perfect",
         artist: "Ed Sheeran"
      },
      {
         title: "Kesariya",
```

```
            artist: "Arijit Singh"
        },
        {
            title: "Blinding Lights",
            artist: "The Weeknd"
        }
    ]
};

const server = http.createServer((req, res) => {
    if (req.url === "/playlist" && req.method === "GET") {
        res.writeHead(200, {
            "Content-Type": "application/json"
        });
        res.end(JSON.stringify(playlist, null, 2));
    } else {
        res.writeHead(404, {
            "Content-Type": "application/json"
        });
        res.end(JSON.stringify({
            error: "Playlist endpoint not found"
        }));
    }
});

server.listen(3000, "0.0.0.0", () => {
    console.log("Now Playing service running on port 3000");
});
```

Save and exit.

Step 3: Start the Node.js service

Run:

```
node /opt/now-playing/app.js
```

You should see:

Now Playing service running on port 3000

Keep this terminal open while testing.

Open another terminal and run:

curl http://localhost:3000/playlist

Expected response:


```
{
  "app": "Now Playing",
  "playlist": [
    {
      "title": "Perfect",
      "artist": "Ed Sheeran"
    },
    {
      "title": "Kesariya",
      "artist": "Arijit Singh"
    },
    {
      "title": "Blinding Lights",
      "artist": "The Weeknd"
    }
  ]
}
```


Step 4: Allow access to port 3000

If you want to access the API from another machine, allow the port through firewalld:

firewall-cmd --permanent --add-port=3000/tcp

firewall-cmd --reload

This is an additional firewall exception to the HTTP and SSH rules from your previous task. Only expose port 3000 to a trusted lab network.

Step 5: Test both services in your browser

PHP Foodie Blog

http://SERVER_IP/index.php

Expected: Welcome to My Foodie Blog

Node.js playlist API

http://SERVER_IP:3000/playlist

Expected: JSON containing the playlist and artist names

Replace SERVER_IP with your server's actual IP address. If you use the browser on the Linux server itself, http://localhost/index.php and http://localhost:3000/playlist will work.

Optional: Keep Node.js running in the background

Once the basic test succeeds, you can create a systemd service.

vi /etc/systemd/system/now-playing.service

Add:

[Unit]

Description=Now Playing Node.js Service

After=network.target

[Service]

Type=simple

WorkingDirectory=/opt/now-playing

ExecStart=/usr/bin/node /opt/now-playing/app.js

Restart=on-failure

[Install]

WantedBy=multi-user.target

Save and run:

systemctl daemon-reload

systemctl enable --now now-playing

systemctl status now-playing

The service should show active (running).

Task 5: Restrict a directory to localhost only

We'll create /var/www/html/secret and configure Apache to allow access only from 127.0.0.1.

Step 1: Create a secret directory and sample page

mkdir -p /var/www/html/secret

echo "This page is accessible only from localhost." > /var/www/html/secret/index.html

restorecon -Rv /var/www/html/secret

Step 2: Edit Apache configuration

Open:

vi /etc/httpd/conf/httpd.conf

Add this block at the end of the file:

<Directory "/var/www/html/secret">

    AllowOverride None

    Require ip 127.0.0.1

</Directory>

This rule permits requests originating from IPv4 localhost only. Requests from other client IP addresses should be denied.

Save and exit.

Step 3: Validate and restart Apache

httpd -t

Expected output:

Syntax OK

Restart Apache:

systemctl restart httpd

Step 4: Test the restriction

On the Linux server, run:

curl -i http://127.0.0.1/secret/

Expected result:

HTTP/1.1 200 OK

Then try:

curl -i http://SERVER_IP/secret/

When this request is made locally to the server's non-loopback IP, it will normally be denied because its source address isn't 127.0.0.1. From another machine, test the same URL using the server IP; it should return 403 Forbidden.